AI Engineer Roadmap - 40-Week Day-by-Day Planner

Built for: junior developer, 4 hrs/day on weekdays, targeting Google / Meta / Microsoft / Amazon / Zomato / Blinkit-type product companies. Print double-sided or one week per sheet and tick boxes as you go.

How a day works (4 hours, Mon-Fri)

Block C - DSA (1h, or 2h in W35-40)	Do it FIRST while fresh so it never gets skipped. 25-30 min attempt, then read/watch the solution, write the pattern in your notebook, redo in 3 days. Log failed problems for spaced revision.
Block A - Learn (2h)	Deep focus: course/docs/book with a resource named in brackets. Take notes in your own words. No phone. If it is a build-sprint day, A and B merge into building.
Block B - Build/Practice (1h)	Apply what you learned the same day in code. Commit to GitHub daily. If you cannot build it, you have not learned it yet.
Saturday (optional 2-3h)	Weekly review, catch up, polish the project, update the tracker. **Sunday**: off. Rest is part of the plan.

Rules that make this work

1. If you fall behind, do NOT compress: extend the week. A finished week beats a rushed one.
2. Every phase ends with a public deliverable (repo + README + demo). Build the portfolio as you learn.
3. The plan uses LeetCode's free problems and avoids premium-only ones.
4. Older LangChain tutorials are outdated: LangChain/LangGraph reached 1.0 in Oct 2025 and old chains moved to langchain-classic. Always check the docs version.
5. Udemy courses are an OPTIONAL second explanation (see the orange box on each week page and the Udemy index near the end). Resource names/course contents can change. If a link or course is gone, search the topic in the bracket and pick an equivalent.
6. Cloud accounts, API keys, job applications and referral messages are yours to create and send. Never paste secrets into code or chat.

Phase map

1	**Python, Software Eng. & FastAPI**	W1-6
2	**Math, ML & Deep Learning basics**	W7-10
3	**LLM App Engineering**	W11-14
4	**RAG & Vector Databases**	W15-20
5	**Agents: LangChain, LangGraph, MCP**	W21-26
6	**Hugging Face, Fine-tuning & Serving**	W27-30
7	**Production, Cloud & Flagship Project**	W31-34
8	**Interview Sprint (DSA, LLD, HLD, mocks)**	W35-40

WEEK 1

PHASE 1 / 8 - PYTHON, SOFTWARE ENG. & FASTAPI

Setup + Python core refresh

Week of: _____ / _____
Hours done: ____ / 20

GOAL: A clean dev environment, Git fluency, and tight Python fundamentals (you already know some, so move fast but write code daily).

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Dev setup: install Python 3.12+, uv, VS Code, ruff. Git basics [Pro Git book ch 1-3 | learngitbranching.js.org first 3 levels]	Create GitHub repo 'ai-eng-journey'; commit README + daily-log.md; add pytest 'hello world' + ruff config	Setup: LeetCode + NeetCode accounts. Big-O primer [NeetCode 'Big-O' lesson]. Solve: Contains Duplicate, Valid Anagram	☐ A ☐ B ☐ C
TUE	Core types, slicing, mutability, truthiness, f-strings, loops [CS50P lectures 0-2 | docs.python.org/3/tutorial ch 3-5]	10 small exercises [Exercism Python track: 'Easy' set]; commit each	Two Sum, Group Anagrams. Learn the hashmap 'complement' pattern	☐ A ☐ B ☐ C
WED	Functions: args/kwargs, scope, closures, lambda, modules & imports [CS50P lecture 3-4 | Real Python 'Defining Your Own Python Function']	Write utils.py (5 helpers) + pytest tests; make it importable as a package	Top K Frequent Elements (bucket sort), Product of Array Except Self	☐ A ☐ B ☐ C
THU	list/dict/set/tuple deep dive, comprehensions, collections (defaultdict, Counter, deque), sorted(key=) [Python docs 'collections' | Real Python]	Word-frequency CLI: read a text file, print top N words (use Counter, argparse)	Valid Sudoku, Sort Characters By Frequency	☐ A ☐ B ☐ C
FRI	Files, JSON, exceptions, context managers (with), pathlib, argparse [Python tutorial ch 7-8 | Real Python 'pathlib']	CLI expense tracker: add/list/delete, saved to JSON, custom exceptions, tests	Longest Consecutive Sequence. Re-solve Two Sum + Group Anagrams from memory	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Weekly review: update daily-log.md, redo missed exercises, write 5 flashcards (Anki) on this week's gotchas.			☐ R

UDEMY (optional supplement)
Learn: U1 Portilla Python: fast-forward through basics; watch only sections on functions, modules, error handling, file I/O if a concept feels shaky.
DSA: U5 DSA: sections on Big-O, arrays and hash tables (before Mon-Wed problems).

WEEKLY DELIVERABLE

[ ] GitHub repo 'ai-eng-journey' with daily-log.md + CLI expense tracker (JSON storage, tested with pytest).

SELF-CHECK: I CAN...

[ ] Explain mutable vs immutable, and why default list args are a bug
[ ] Use venv/uv, git branch, commit, PR without help
[ ] Write a function with *args/**kwargs and a closure

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 2

PHASE 1 / 8 - PYTHON, SOFTWARE ENG. & FASTAPI

Intermediate Python: OOP, typing, generators, decorators

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Move from scripts to well-structured Python: classes, type hints, generators, decorators. This is the code style AI libraries use.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Classes: __init__, instance vs class attributes, dunder methods [Corey Schafer OOP playlist vids 1-6 | Fluent Python ch 1]	Model Task and TaskList classes with __repr__/__eq__/__len__	Valid Palindrome, Two Sum II (sorted array) - two-pointer template	☐ A ☐ B ☐ C
TUE	Inheritance vs composition, ABCs, @property, @classmethod/@staticmethod [Real Python 'OOP in Python 3' | Fluent Python ch 14]	Refactor yesterday's code to favour composition; add a Storage abstract base class	3Sum, Remove Duplicates from Sorted Array	☐ A ☐ B ☐ C
WED	Dataclasses + typing (Optional, Union, list[str], TypedDict, Protocol); mypy/pyright [docs.python.org typing | Real Python 'Python Type Checking']	Convert models to dataclasses; add type hints everywhere; get mypy to pass	Container With Most Water, Move Zeroes	☐ A ☐ B ☐ C
THU	Iterators & generators: yield, itertools, lazy evaluation [Fluent Python ch 17 | Corey Schafer 'Generators']	Log-file processor that streams a large file line-by-line with generators	Trapping Rain Water (two pointers) - take your time, re-derive it	☐ A ☐ B ☐ C
FRI	Decorators, closures, functools (wraps, lru_cache), custom context managers [Real Python 'Primer on Python Decorators']	Write @timer and @retry(times, backoff) decorators + tests	Sort Colors (Dutch flag), 3Sum Closest. Redo 3Sum cold	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Polish CLI Task Manager, add pytest-cov report, tag v0.1. Review the week's flashcards.			☐ R

UDEMY (optional supplement)
Learn: U1 Portilla Python: sections on OOP, decorators, generators (watch at 1.25x after reading Fluent Python / Corey Schafer).
DSA: U5 DSA: two-pointer / sliding-window style exercises and string handling.

WEEKLY DELIVERABLE

[ ] 'CLI Task Manager' refactored with classes, dataclasses, full type hints (mypy clean), generators and decorators; 80%+ test coverage.

SELF-CHECK: I CAN...

[ ] Explain __init__, __repr__, __eq__ and when to use dataclass
[ ] Write a decorator with arguments using functools.wraps
[ ] Explain generator vs list memory behaviour

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 3

PHASE 1 / 8 - PYTHON, SOFTWARE ENG. & FASTAPI

Testing, Pydantic, HTTP and asyncio

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Testing habits, data validation with Pydantic, HTTP fundamentals, and async programming (essential for LLM apps).

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	pytest: fixtures, parametrize, monkeypatch/mock [pytest docs 'Get Started' + 'How-to fixtures' | Real Python 'Effective Python Testing with pytest']	Raise coverage of W2 project to 85% with pytest-cov	Best Time to Buy and Sell Stock, Maximum Average Subarray I	☐ A ☐ B ☐ C
TUE	HTTP + REST fundamentals: methods, status codes, headers, JSON, timeouts [MDN 'An overview of HTTP' | httpx docs quickstart]	Script calling the GitHub REST API with httpx, timeouts and retry logic	Longest Substring Without Repeating Characters - fixed vs variable window	☐ A ☐ B ☐ C
WED	Pydantic v2: BaseModel, Field, validators, model_dump, pydantic-settings, .env [docs.pydantic.dev Concepts]	Define Pydantic models for GitHub responses + a Settings class reading .env	Longest Repeating Character Replacement, Permutation in String	☐ A ☐ B ☐ C
THU	Concurrency: threads vs processes vs asyncio, the GIL, event loop [Real Python 'Async IO in Python: A Complete Walkthrough' | asyncio docs overview]	Fetch 50 URLs sync vs async; measure and record the speed-up	Minimum Window Substring (hard) - read editorial, then code	☐ A ☐ B ☐ C
FRI	asyncio deep dive: create_task, gather, Semaphore, timeouts, async context managers/generators [asyncio docs 'Coroutines and Tasks']	Rate-limited async fetcher using Semaphore; handle cancellations and errors	Sliding Window Maximum (deque). Redo Min Window Substring	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Assemble 'Async GitHub Repo Analyzer' CLI (top repos by stars, languages, activity). README + tests.			☐ R

UDEMY (optional supplement)
Learn: No Udemy course needed this week - the free resources in Block A are the best fit. (pytest/asyncio are better from docs + Real Python.)
DSA: U5 DSA: any hashing/strings exercises you still find hard.

WEEKLY DELIVERABLE

[ ] 'Async GitHub Repo Analyzer' CLI: concurrent API calls, Pydantic models, retries, tests with mocked HTTP.

SELF-CHECK: I CAN...

[ ] Explain what the event loop does
[ ] Write a pytest fixture + parametrized test
[ ] Validate untrusted JSON with Pydantic

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 4

PHASE 1 / 8 - PYTHON, SOFTWARE ENG. & FASTAPI

FastAPI fundamentals (coming from Flask)

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Get productive in FastAPI: routing, Pydantic bodies, dependency injection, structure, middleware, and streaming (needed for LLM output).

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	FastAPI first steps: ASGI vs WSGI, uvicorn, path/query params, auto-docs [fastapi.tiangolo.com/tutorial: First Steps -> Query Parameters]	Port 1-2 of your old Flask endpoints to FastAPI; note the differences in a Flask-vs-FastAPI.md	Binary Search, Search Insert Position - write the template without bugs	☐ A ☐ B ☐ C
TUE	Request bodies, response_model, status codes, validation errors [FastAPI tutorial: Body, Response Model, Handling Errors]	In-memory CRUD notes API with Pydantic schemas (create/read/update/delete)	Search a 2D Matrix, Koko Eating Bananas (search on answer)	☐ A ☐ B ☐ C
WED	Dependencies (Depends), APIRouter, project layout [FastAPI tutorial: Dependencies, Bigger Applications]	Restructure into routers/ services/ schemas/ core/config.py	Find Minimum in Rotated Sorted Array, Search in Rotated Sorted Array	☐ A ☐ B ☐ C
THU	Middleware, CORS, exception handlers, BackgroundTasks, lifespan events [FastAPI docs: Middleware, Background Tasks, Lifespan]	Add global exception handler + request-ID logging middleware	Time Based Key-Value Store, First Bad Version	☐ A ☐ B ☐ C
FRI	async def vs def endpoints; StreamingResponse and Server-Sent Events [FastAPI docs: Custom Response | sse-starlette README]	Endpoint that streams a counter every 0.5s via SSE; consume with httpx stream	Median of Two Sorted Arrays (hard) - attempt + study. Redo rotated array	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Write 1-page 'Flask vs FastAPI' comparison; review. Push notes API v0.1.			☐ R

UDEMY (optional supplement)
Learn: U2 Roby FastAPI: intro + first project, HTTP methods, path/query params, request validation with Pydantic, Swagger examples, status codes/error handling.
DSA: U5 DSA: searching and sorting sections (binary search before Tue).

WEEKLY DELIVERABLE

[ ] Notes API with CRUD, routers/services/schemas structure, error handling, request-ID logging middleware, SSE streaming endpoint.

SELF-CHECK: I CAN...

[ ] Explain ASGI vs WSGI and when async def matters
[ ] Use Depends() for DB/session/auth injection
[ ] Stream a response with SSE

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 5

PHASE 1 / 8 - PYTHON, SOFTWARE ENG. & FASTAPI

Databases, SQLAlchemy and auth

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Persistent data + security: SQL, PostgreSQL, SQLAlchemy 2.0, migrations, JWT auth.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	SQL fundamentals: SELECT, JOIN, GROUP BY, subqueries [sqlbolt.com lessons 1-18 | Mode SQL Tutorial intermediate]	Write 20 queries on a sample DB (postgresqltutorial.com 'dvdrental')	Valid Parentheses, Min Stack	☐ A ☐ B ☐ C
TUE	PostgreSQL + Docker, schema design, normalization, FKs, indexes, EXPLAIN [postgresqltutorial.com | Docker docs 'Get started']	Design users/notes/tags schema; run Postgres with docker compose	Evaluate Reverse Polish Notation, Daily Temperatures (monotonic stack)	☐ A ☐ B ☐ C
WED	SQLAlchemy 2.0 ORM: models, sessions, relationships, queries [SQLAlchemy 2.0 'Unified Tutorial']	Replace in-memory storage with Postgres via SQLAlchemy	Car Fleet, Next Greater Element I	☐ A ☐ B ☐ C
THU	Alembic migrations; async SQLAlchemy with asyncpg; transactions [Alembic tutorial | SQLAlchemy asyncio docs]	Add migrations; async session dependency; transaction handling	Largest Rectangle in Histogram (hard-ish), Implement Queue using Stacks	☐ A ☐ B ☐ C
FRI	Auth: password hashing (argon2/bcrypt), JWT, OAuth2PasswordBearer [FastAPI tutorial: Security]	Register/login endpoints; protect routes; per-user notes	Online Stock Span, Backspace String Compare. Redo Daily Temperatures	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** LeetCode SQL 50 study plan: solve 10 problems. Review + push.			☐ R

UDEMY (optional supplement)
Learn: U2 Roby FastAPI: authentication/authorization and production-database sections. U3 SQL Bootcamp: SELECT/GROUP BY, JOINs, creating databases and tables (Mon-Tue).
DSA: U5 DSA: stack and queue sections.

WEEKLY DELIVERABLE

[ ] Notes API backed by PostgreSQL with Alembic migrations, async sessions, user registration/login (JWT), per-user notes.

SELF-CHECK: I CAN...

[ ] Write JOIN/GROUP BY queries without help
[ ] Explain an index and when it helps
[ ] Explain JWT flow and password hashing

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 6

PHASE 1 / 8 - PYTHON, SOFTWARE ENG. & FASTAPI

Docker, Redis, CI/CD, finishing the service

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Make the service production-style: containerized, cached, tested end-to-end, CI-checked, documented.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Docker deep dive: Dockerfile, layers, multi-stage builds, compose, volumes, env vars [Docker docs 'Get started' + 'Best practices']	Dockerize API + Postgres with docker compose; one command to run everything	Reverse Linked List, Merge Two Sorted Lists	☐ A ☐ B ☐ C
TUE	Redis: caching patterns, TTL, rate limiting, redis-py [redis.io 'Get started' | Redis University RU101]	Cache GET /notes (cache-aside) and add a per-user rate limiter	Linked List Cycle (Floyd), Reorder List	☐ A ☐ B ☐ C
WED	API testing: TestClient/httpx AsyncClient, test DB, fixtures, dependency overrides [FastAPI tutorial: Testing]	Reach 25+ tests and 80%+ coverage	Remove Nth Node From End, Add Two Numbers	☐ A ☐ B ☐ C
THU	GitHub Actions: lint, type-check, test, docker build [GitHub Actions 'Quickstart' + 'Building and testing Python']	Add CI workflow; badge in README; fix failures	Copy List with Random Pointer, Find the Duplicate Number	☐ A ☐ B ☐ C
FRI	Structured logging, 12-factor config, Makefile, API docs polish [12factor.net | structlog/loguru docs]	Finish README (architecture diagram), tag v1.0, record a 2-min demo	LRU Cache (hashmap + DLL) - this is asked a LOT. Merge k Sorted Lists (preview)	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** PHASE 1 RETRO: what was hard? update daily-log, fix gaps. Deliverable check: repo public, README solid.			☐ R

UDEMY (optional supplement)
Learn: U4 Docker Mastery: containers, images, volumes, Compose, Docker in production, security, GitHub Actions CI lectures (Mon + Thu). U2: deployment section.
DSA: U5 DSA: linked list + doubly linked list sections and their exercises (before Mon-Tue problems).

WEEKLY DELIVERABLE

[ ] PROJECT 1: 'Production-style FastAPI service' (Dockerized, Postgres, Redis, 25+ tests, GitHub Actions CI, README with architecture diagram, 2-min demo).

SELF-CHECK: I CAN...

[ ] Explain image vs container and multi-stage builds
[ ] Describe cache-aside pattern and TTL
[ ] Show a green CI pipeline

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 7

PHASE 2 / 8 - MATH, ML & DEEP LEARNING BASICS

Math essentials + NumPy/Pandas

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Just enough linear algebra, calculus, probability and data tooling to understand how models learn.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Linear algebra I: vectors, dot product, matrices [3Blue1Brown 'Essence of Linear Algebra' ep 1-4]	Implement dot product and matmul in pure Python; verify vs NumPy	Invert Binary Tree, Maximum Depth of Binary Tree - recursion on trees	☐A ☐B ☐C
TUE	Linear algebra II: linear transformations, determinant, eigen-intuition [3Blue1Brown ep 5-8, 14]	Cosine similarity from scratch + NumPy vectorized version (you'll reuse for embeddings)	Diameter of Binary Tree, Balanced Binary Tree	☐A ☐B ☐C
WED	Calculus: derivatives, chain rule, partial derivatives [3Blue1Brown 'Essence of Calculus' ep 1-4 | Khan Academy]	Numerically estimate gradients; run gradient descent on f(x,y)	Same Tree, Subtree of Another Tree	☐A ☐B ☐C
THU	Probability & statistics: distributions, mean/variance, conditional prob, Bayes [StatQuest 'Statistics Fundamentals']	Simulate dice/coin; verify the Central Limit Theorem in NumPy	Binary Tree Level Order Traversal (BFS), Binary Tree Right Side View	☐A ☐B ☐C
FRI	NumPy + Pandas: arrays, broadcasting, groupby, merge [Kaggle Learn: Pandas | NumPy 'Absolute beginners' guide]	EDA on Titanic (Kaggle): missing values, groupbys, 3 plots	Count Good Nodes in Binary Tree, Path Sum. Redo Diameter	☐A ☐B ☐C
SAT	**Optional 2-3h: review, catch-up, project polish.** Revisit weakest math topic via Khan Academy; commit notebook.			☐R

UDEMY (optional supplement)
Learn: No Udemy course needed this week - the free resources in Block A are the best fit.
DSA: U5 DSA: tree sections (traversals, BST) before Mon problems.

WEEKLY DELIVERABLE

[ ] Notebook: from-scratch dot/matmul, cosine similarity, gradient descent, CLT simulation, and an EDA on a Kaggle dataset.

SELF-CHECK: I CAN...

[ ] Explain dot product and cosine similarity geometrically
[ ] Explain what a gradient is and why we step opposite to it
[ ] Explain mean/variance/Bayes in plain words

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 8

PHASE 2 / 8 - MATH, ML & DEEP LEARNING BASICS

Classical machine learning

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Core ML ideas you will be asked about in interviews: generalization, metrics, models, evaluation hygiene.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	ML foundations: train/val/test, bias-variance, overfitting, linear regression [Andrew Ng ML Specialization Course 1 wk 1-2 | StatQuest 'Machine Learning' playlist]	Linear regression from scratch with gradient descent; compare to sklearn	Validate Binary Search Tree, Kth Smallest Element in a BST	☐☐☐ A B C
TUE	Logistic regression + classification metrics: precision, recall, F1, ROC-AUC [Ng Course 1 wk 3 | StatQuest ROC & AUC]	sklearn Pipeline on Titanic with metrics report	Lowest Common Ancestor of a BST, Lowest Common Ancestor of a Binary Tree	☐☐☐ A B C
WED	Decision trees, random forests, gradient boosting [StatQuest 'Decision Trees', 'XGBoost' | scikit-learn user guide 1.10, 1.11]	Cross-validation + hyperparameter search; compare 3 models	Construct Binary Tree from Preorder and Inorder Traversal	☐☐☐ A B C
THU	Unsupervised: k-means, PCA; embeddings intuition [StatQuest 'PCA', 'K-means' | scikit-learn clustering guide]	Cluster + visualize; PCA on TF-IDF of news text	Binary Tree Maximum Path Sum (hard)	☐☐☐ A B C
FRI	Feature engineering, leakage, evaluation hygiene [Kaggle Learn 'Intermediate ML' + 'Feature Engineering']	Submit to a Kaggle practice competition; write 1-page evaluation report	Serialize and Deserialize Binary Tree (hard). Redo Validate BST	☐☐☐ A B C
SAT	**Optional 2-3h: review, catch-up, project polish.** Make a 1-page ML interview cheat-sheet (bias/variance, metrics, regularization, trees vs linear, leakage).			☐ R

UDEMY (optional supplement)
Learn: U7 ML A-Z: regression, classification, clustering, model-selection sections (hands-on coding; Ng/StatQuest for the 'why').
DSA: U5 DSA: BST + recursion exercises.

WEEKLY DELIVERABLE

[ ] Kaggle-style mini project (e.g. Titanic or house prices): scikit-learn pipeline, CV tuning, written evaluation report.

SELF-CHECK: I CAN...

[ ] Explain bias-variance and overfitting
[ ] Pick precision/recall/F1/AUC for a given problem
[ ] Explain data leakage with an example

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 9

PHASE 2 / 8 - MATH, ML & DEEP LEARNING BASICS

Neural nets from scratch + PyTorch

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Understand backprop by building it, then learn PyTorch's training loop.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Neurons and backprop [3Blue1Brown 'Neural Networks' ch 1-4 | Karpathy 'The spelled-out intro to neural networks and backpropagation: building micrograd' part 1]	Follow along; code the Value class with autograd	Kth Largest Element in a Stream, Last Stone Weight - heapq basics	☐ A ☐ B ☐ C
TUE	micrograd part 2: build a small MLP, train it [Karpathy micrograd, rest]	Train your autograd MLP on a toy dataset; plot the loss	K Closest Points to Origin, Kth Largest Element in an Array	☐ A ☐ B ☐ C
WED	PyTorch basics: tensors, autograd, nn.Module, optimizers, DataLoader [pytorch.org/tutorials 'Learn the Basics' parts 1-6]	MNIST classifier (MLP) with a clean training loop	Task Scheduler, Top K Frequent Words	☐ A ☐ B ☐ C
THU	Training mechanics: loss functions, regularization, dropout, batch norm, LR schedules, using a GPU on Colab [PyTorch tutorials | CS231n notes on training]	Upgrade to a CNN, hit > 98%; log accuracy curves	Design Twitter, Reorganize String	☐ A ☐ B ☐ C
FRI	Karpathy 'The spelled-out intro to language modeling: building makemore' (part 1: bigrams)	Reproduce the notebook; generate names from your model	Find Median from Data Stream (hard, two heaps). Redo K Closest Points	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Review gradient math; tidy repo; flashcards on PyTorch API.			☐ R

UDEMY (optional supplement)
Learn: U8 PyTorch Bootcamp: PyTorch fundamentals, workflow, neural-network classification sections (alongside Karpathy's micrograd).
DSA: U5 DSA: heap / priority-queue section before Mon.

WEEKLY DELIVERABLE

[ ] Your own micrograd-style autograd engine + PyTorch MNIST CNN > 98% accuracy + makemore bigram model.

SELF-CHECK: I CAN...

[ ] Explain backprop with the chain rule on a 2-layer net
[ ] Write a PyTorch training loop from memory
[ ] Explain train vs eval mode, dropout, batch norm

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 10

PHASE 2 / 8 - MATH, ML & DEEP LEARNING BASICS

Language models and transformers

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Demystify how GPT-style models work: tokenization, embeddings, attention, training a tiny GPT.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	makemore parts 2-3: MLP language model, embeddings, activations/batchnorm [Karpathy Zero to Hero: makemore 2 & 3]	Reproduce the notebooks; experiment with embedding size and context length	Subsets, Combination Sum - the backtracking template	☐ A ☐ B ☐ C
TUE	Tokenization & embeddings, BPE [Karpathy 'Let's build the GPT Tokenizer' (first half) | HF LLM Course ch 2]	Compare tiktoken vs HF tokenizers on English, Hindi, and code; count tokens/cost	Permutations, Subsets II	☐ A ☐ B ☐ C
WED	Attention & transformer architecture [Jay Alammar 'The Illustrated Transformer' | 3Blue1Brown 'Attention in transformers']	Implement scaled dot-product attention in PyTorch with a mask	Combination Sum II, Generate Parentheses	☐ A ☐ B ☐ C
THU	Karpathy 'Let's build GPT: from scratch, in code, spelled out' - first half	Follow along: bigram baseline -> self-attention	Word Search, Letter Combinations of a Phone Number	☐ A ☐ B ☐ C
FRI	'Let's build GPT' second half + Karpathy 'Intro to Large Language Models' (1-hr talk)	Train a tiny GPT on a text corpus; write 'How an LLM works' note	Palindrome Partitioning. Redo Subsets + Permutations without notes	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** PHASE 2 RETRO: push tiny-GPT repo, polish ML cheat-sheet. Start an 'LLM interview questions' doc (add ~5 per week from here).			☐ R

UDEMY (optional supplement)
Learn: No Udemy course needed this week - the free resources in Block A are the best fit. (Karpathy + Alammar are the primary resources.)
DSA: U5 DSA: recursion and backtracking-style exercises.

WEEKLY DELIVERABLE

[ ] PROJECT 2: Tiny GPT trained from scratch on a text corpus + 1-page 'How an LLM works' note in your own words.

SELF-CHECK: I CAN...

[ ] Explain attention (Q, K, V) in simple terms
[ ] Explain BPE tokenization and why tokens matter for cost
[ ] Explain pretraining vs fine-tuning vs RLHF at a high level

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____

_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 11

PHASE 3 / 8 - LLM APP ENGINEERING

First LLM API calls, prompting, structured output

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Call LLM APIs directly (no frameworks yet), write solid prompts, and get reliable structured JSON back.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	LLM API fundamentals: messages, roles, temperature, max tokens, stop sequences, tokens and pricing [Anthropic API docs 'Getting started' | OpenAI API docs quickstart]	Call two providers via their SDKs; write a thin provider-agnostic wrapper class	N-Queens (hard) - study the pruning idea	☐☐☐ A B C
TUE	Prompt engineering: clear instructions, system prompts, few-shot, XML/delimiters, step-by-step reasoning [Anthropic prompt engineering overview | DeepLearning.AI 'ChatGPT Prompt Engineering for Developers']	5 prompt tasks (summarize, extract, classify, rewrite, reason); keep v1/v2/v3 and compare outputs	Implement Trie (Prefix Tree)	☐☐☐ A B C
WED	Structured output: JSON schemas / tool-use for extraction, Pydantic validation, retry on invalid [provider docs 'Structured outputs' / 'Tool use' | Instructor library docs]	Resume or invoice extractor -> Pydantic model with automatic retry	Design Add and Search Words Data Structure	☐☐☐ A B C
THU	Streaming: token-by-token output, async clients [SDK docs 'Streaming']	FastAPI SSE endpoint streaming LLM tokens; consume with a tiny HTML page	Word Search II (trie + backtracking, hard)	☐☐☐ A B C
FRI	Cost and latency: token counting, prompt caching, batch APIs, picking models [Anthropic docs 'Prompt caching' | Chip Huyen 'AI Engineering' ch 1-2]	Middleware logging tokens, cost, latency per request into Postgres	Redo: Word Search, Combination Sum II. Mixed review of failed problems	☐☐☐ A B C
SAT	**Optional 2-3h: review, catch-up, project polish.** Read AI Engineering ch 1-2 notes; tidy wrapper; flashcards on LLM terms.			☐ R

UDEMY (optional supplement)
Learn: U9 Ed Donner Core Track: Week 1 (first LLM product, frontier model APIs, prompting).
DSA: U5 DSA: trie / recursion exercises.

WEEKLY DELIVERABLE

[ ] Provider-agnostic LLM client wrapper + invoice/resume extractor returning validated Pydantic models + FastAPI streaming endpoint.

SELF-CHECK: I CAN...

[ ] Explain tokens, context window, temperature
[ ] Write a system prompt with examples and delimiters
[ ] Enforce a JSON schema and retry on invalid output

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 12

PHASE 3 / 8 - LLM APP ENGINEERING

Tool calling, memory, multimodal, safety

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Make the model act and remember: function calling, conversation state, vision/PDF input, basic security.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Tool / function calling mechanics [Anthropic 'Tool use' docs | OpenAI function calling guide]	Raw-API tools: calculator, weather (public API), DB lookup	Number of Islands, Max Area of Island - grid DFS/BFS	☐A ☐B ☐C
TUE	Multi-turn conversations: state, sliding window, summarization memory, context limits [provider docs 'Messages' | AI Engineering ch on context]	Persistent chat sessions in Redis/Postgres with trimming/summarization	Clone Graph, Flood Fill	☐A ☐B ☐C
WED	Multimodal input: images and PDFs [provider docs 'Vision' / 'PDF support']	Receipt/PDF -> JSON extraction endpoint	Pacific Atlantic Water Flow, Surrounded Regions	☐A ☐B ☐C
THU	Safety basics: prompt injection, jailbreaks, PII, output validation [OWASP Top 10 for LLM Applications | provider safety docs]	Write 20 injection test cases; add input/output validators	Rotting Oranges (multi-source BFS), Find if Path Exists in Graph	☐A ☐B ☐C
FRI	Open models locally: Ollama, OpenAI-compatible endpoints, swapping providers [ollama.com docs | llama.cpp README overview]	Add a local model as a provider behind your wrapper; compare quality/speed	Shortest Path in Binary Matrix. Redo Number of Islands cold	☐A ☐B ☐C
SAT	**Optional 2-3h: review, catch-up, project polish.** Review security notes; push code; read AI Engineering chapter on prompt engineering.			☐R

UDEMY (optional supplement)
Learn: U9 Ed Donner Core Track: the weeks covering multi-model chat UIs and tool use (check curriculum titles).
DSA: U5 DSA: graph sections (BFS/DFS) before Mon.

WEEKLY DELIVERABLE

[ ] Chat backend with 3 tools, persistent conversation memory, PDF/image extraction endpoint, injection test-suite, and a local-model provider.

SELF-CHECK: I CAN...

[ ] Walk through the tool-calling round trip
[ ] Name 3 context-management strategies
[ ] Describe prompt injection and 2 mitigations

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 13

PHASE 3 / 8 - LLM APP ENGINEERING

Build sprint: LLM chat service

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Combine everything into one real service with auth, tools, limits and tests.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Design first: requirements, architecture diagram, API contract [AI Engineering: chapter on AI engineering architecture]	Write a 1-page design doc + OpenAPI skeleton	Course Schedule (cycle detection, topo sort)	☐ A ☐ B ☐ C
TUE	Simple UI that streams: Streamlit or Gradio or plain HTML + JS EventSource [Streamlit 'chat' docs | MDN EventSource]	Chat UI with streaming and message history	Course Schedule II, Find Eventual Safe States	☐ A ☐ B ☐ C
WED	Build sprint: reuse Project 1 auth; per-user conversations in Postgres	Auth + conversation CRUD + tests	Number of Provinces, Redundant Connection (union-find)	☐ A ☐ B ☐ C
THU	Build sprint: tools + guardrails: 2 tools, rate limiting (Redis), per-user cost cap	Implement and test the limits	Word Ladder (BFS, hard-ish), Is Graph Bipartite?	☐ A ☐ B ☐ C
FRI	Testing LLM apps: mock the LLM, golden tests, contract tests	Mocked-LLM test suite + docker compose up works end to end	Accounts Merge. Redo Course Schedule II	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Polish; write README with architecture diagram.			☐ R

UDEMY (optional supplement)
Learn: No Udemy course needed this week - the free resources in Block A are the best fit.
DSA: U5 DSA: graph exercises; topological sort + union-find.

WEEKLY DELIVERABLE

[ ] Chat app: FastAPI backend + simple UI, auth, per-user conversations, 2+ tools, per-user rate limit and cost cap, mocked-LLM tests, docker compose.

SELF-CHECK: I CAN...

[ ] Draw your architecture on a whiteboard
[ ] Explain how you test non-deterministic LLM code
[ ] Show cost per conversation

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 14

PHASE 3 / 8 - LLM APP ENGINEERING

Evals 101, tracing, deploy

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Start measuring quality: eval sets, LLM-as-judge, tracing. Then deploy.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Why evals matter; building a test set; rule-based checks [Hamel Husain 'Your AI Product Needs Evals' | Eugene Yan 'Task-specific LLM evals']	Create a 30-case eval set for your chatbot's tasks (with expected behaviours)	Network Delay Time (Dijkstra) - implement with heapq	☐A ☐B ☐C
TUE	LLM-as-judge: rubrics, position/verbosity bias, calibrating with human labels [Eugene Yan 'Evaluating LLM-evaluators' | DeepLearning.AI 'Evaluating and Debugging Generative AI']	Write a judge prompt; measure agreement with 30 human labels	Cheapest Flights Within K Stops (Bellman-Ford / BFS)	☐A ☐B ☐C
WED	Tracing/observability: Langfuse quickstart [langfuse.com/docs]	Trace every LLM call; inspect 10 traces and note failures	Path With Minimum Effort, Swim in Rising Water	☐A ☐B ☐C
THU	Deploy: container to a simple host (Render / Railway / Fly.io) [host docs]	Deploy the service; set secrets; smoke test	Min Cost to Connect All Points (Prim / Kruskal)	☐A ☐B ☐C
FRI	Write-up day: blog post #1 'What I learned building an LLM chat service' (dev.to / Medium / personal)	Publish; add link to GitHub README; share on LinkedIn	Reconstruct Itinerary (hard). Redo Network Delay Time	☐A ☐B ☐C
SAT	**Optional 2-3h: review, catch-up, project polish.** PHASE 3 RETRO: gaps list, update interview-questions doc, review costs.			☐R

UDEMY (optional supplement)
Learn: U9 Ed Donner Core Track: Week 4 'LLM Showdown: Evaluating Models' for evaluation ideas.
DSA: U5 DSA: shortest path (Dijkstra) and priority-queue graph problems.

WEEKLY DELIVERABLE

[ ] PROJECT 3: Deployed LLM chat service + 30-case eval set + judge prompt + Langfuse tracing + blog post #1.

SELF-CHECK: I CAN...

[ ] Explain why 'looks good' is not an eval
[ ] Describe LLM-as-judge pitfalls (bias, calibration)
[ ] Show traces of your app

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 15

PHASE 4 / 8 - RAG & VECTOR DATABASES

RAG fundamentals: embeddings, parsing, chunking

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Build a no-framework RAG end to end so you understand every moving part.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	RAG concept and pipeline; what embeddings are [LangChain 'RAG From Scratch' videos 1-4 | Pinecone Learning Center 'Retrieval Augmented Generation']	Embed 100 sentences; build NumPy cosine-similarity search	Maximum Subarray (Kadane), Jump Game	☐ A ☐ B ☐ C
TUE	Embedding models: sentence-transformers, dimensions, similarity metrics, MTEB [sbert.net docs | MTEB leaderboard on Hugging Face]	Compare 3 embedding models on a small QA set; record recall@5	Jump Game II, Gas Station	☐ A ☐ B ☐ C
WED	Document loading/parsing: PDFs, HTML, tables, metadata [PyMuPDF docs | Unstructured docs]	Ingest 20 messy PDFs to clean text + metadata (source, page)	Hand of Straights, Partition Labels, Merge Triplets to Form Target	☐ A ☐ B ☐ C
THU	Chunking strategies: fixed, recursive, semantic, structure-aware, overlap [LangChain text splitters docs | Pinecone 'Chunking strategies']	Evaluate 3 chunkers on retrieval of 20 questions; table of results	Merge Intervals, Insert Interval, Non-overlapping Intervals	☐ A ☐ B ☐ C
FRI	Put it together: naive RAG with FAISS or Chroma, no framework	Working script answering questions with cited sources	Car Pooling, Minimum Number of Arrows to Burst Balloons. Redo Merge Intervals	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Write down failure modes you saw (wrong chunk, missing info). Start a rag-experiments.md log.			☐ R

UDEMY (optional supplement)
Learn: U9 Ed Donner Core Track: the RAG week (embeddings, chunking, vector stores).
DSA: U5 DSA: greedy + intervals exercises.

WEEKLY DELIVERABLE

[ ] Script: ingest messy PDFs -> chunks -> embeddings -> vector search -> grounded answer with citations; chunker comparison table.

SELF-CHECK: I CAN...

[ ] Draw the RAG pipeline (ingest + query paths)
[ ] Explain why chunk size/overlap matter
[ ] Compare 2 embedding models on your data

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 16

PHASE 4 / 8 - RAG & VECTOR DATABASES

Vector databases

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Know the vector DB landscape well enough to choose and defend a design.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	ANN indexes: flat, IVF, HNSW, PQ concepts [Pinecone 'Faiss: The Missing Manual' + HNSW explainer | FAISS wiki]	FAISS flat vs HNSW: measure speed and recall	Climbing Stairs, Min Cost Climbing Stairs, House Robber - DP thinking: state, transition, base	☐ A ☐ B ☐ C
TUE	Chroma + metadata filtering [Chroma docs | DeepLearning.AI 'Advanced Retrieval for AI with Chroma']	Persist collections; filter by metadata; test	House Robber II, Longest Palindromic Substring	☐ A ☐ B ☐ C
WED	pgvector in Postgres: schema, HNSW/IVFFlat indexes, SQL joins with vectors [github.com/pgvector/pgvector README]	Move the corpus to pgvector; hybrid query with SQL filters	Palindromic Substrings, Decode Ways	☐ A ☐ B ☐ C
THU	Qdrant: collections, payload filters, quantization [qdrant.tech/documentation quickstart]	Run Qdrant in Docker; same corpus; compare latency to pgvector	Coin Change, Maximum Product Subarray	☐ A ☐ B ☐ C
FRI	Choosing a vector DB: updates, deletes, multi-tenancy, cost, ops	Write 1-page decision doc: 'Which vector DB for X and why'	Word Break, Longest Increasing Subsequence. Redo Coin Change	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Review; benchmarks table to README.			☐ R

UDEMY (optional supplement)
Learn: U10 Eden Marco LangChain: RAG section on embeddings, vector databases and retrieval (covers Pinecone and FAISS).
DSA: U5 DSA: dynamic programming intro + recursion/memoization.

WEEKLY DELIVERABLE

[ ] Same corpus indexed in FAISS, Chroma, pgvector and Qdrant + latency/recall comparison + 1-page decision doc.

SELF-CHECK: I CAN...

[ ] Explain HNSW at a high level
[ ] Explain exact vs approximate search trade-offs
[ ] State when you would pick pgvector over a dedicated DB

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 17

PHASE 4 / 8 - RAG & VECTOR DATABASES

Retrieval quality: hybrid, rerank, query transformation

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Where RAG quality is won or lost. Improve retrieval measurably.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	BM25/keyword search and hybrid search with Reciprocal Rank Fusion [Qdrant hybrid search docs | rank_bm25 README]	Hybrid retrieval with RRF; compare vs vector-only	Unique Paths, Minimum Path Sum - 2D grid DP	☐ A ☐ B ☐ C
TUE	Reranking with cross-encoders / rerank APIs [sbert.net cross-encoder docs | Cohere or Voyage rerank docs]	Add a reranker; measure the gain and added latency	Longest Common Subsequence (LCS)	☐ A ☐ B ☐ C
WED	Query transformation: rewriting, multi-query, HyDE, step-back [RAG From Scratch videos 5-9]	Add query rewriting + multi-query; test on tricky questions	Partition Equal Subset Sum, Target Sum - 0/1 knapsack family	☐ A ☐ B ☐ C
THU	Contextual retrieval, parent-child, late chunking [Anthropic 'Introducing Contextual Retrieval' post]	Add contextual chunk headers; re-run eval	Coin Change II, Best Time to Buy and Sell Stock with Cooldown	☐ A ☐ B ☐ C
FRI	Retrieval metrics: recall@k, MRR, nDCG; golden set creation [RAGAS docs | Pinecone 'Evaluation measures']	Build 50-question eval set + evaluation script	Interleaving String, Longest Increasing Path in a Matrix. Redo LCS	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Log all experiments in a table (change, metric before/after).			☐ R

UDEMY (optional supplement)
Learn: No Udemy course needed this week - the free resources in Block A are the best fit. (Retrieval quality is best learned from the Qdrant/Sentence-Transformers docs and your own evals.)
DSA: U5 DSA: DP sections (1D).

WEEKLY DELIVERABLE

[ ] Hybrid (BM25 + vector, RRF) + reranker + query rewriting pipeline with a 50-question retrieval eval set and metrics.

SELF-CHECK: I CAN...

[ ] Explain RRF
[ ] Explain bi-encoder vs cross-encoder
[ ] Compute recall@k and MRR

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 18

PHASE 4 / 8 - RAG & VECTOR DATABASES

Generation quality, RAGAS, LangChain v1 for RAG

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Grounded answers, end-to-end RAG evaluation, and first contact with LangChain v1.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Grounded generation: citations, 'I don't know', prompt layout, long-context vs RAG [Anthropic docs 'Long context prompting tips']	Enforce citations in answers; add an abstain rule + tests	Edit Distance	☐A ☐B ☐C
TUE	RAG evaluation with RAGAS: faithfulness, answer relevancy, context precision/recall [docs.ragas.io | DeepLearning.AI 'Building and Evaluating Advanced RAG Applications']	Run RAGAS on your 50 questions; record baseline	Distinct Subsequences, Longest Palindromic Subsequence	☐A ☐B ☐C
WED	LangChain v1 core: models, messages, tools, retrievers (note: pre-1.0 tutorials are outdated; legacy moved to langchain-classic) [docs.langchain.com Python 'Get started' + 'Retrieval']	Rebuild your RAG with LangChain components	Maximal Square, Triangle	☐A ☐B ☐C
THU	LlamaIndex overview for comparison [docs.llamaindex.ai 'Starter tutorial']	Same corpus in LlamaIndex; compare code size, flexibility; choose what you prefer	Burst Balloons (hard) - study interval DP	☐A ☐B ☐C
FRI	Iteration day: attack your worst metric (retrieval or generation)	Run 3 experiments; update experiment log	Regular Expression Matching (hard). Redo Edit Distance	☐A ☐B ☐C
SAT	**Optional 2-3h: review, catch-up, project polish.** Review what actually moved the metrics; flashcards.			☐R

UDEMY (optional supplement)
Learn: U10 Eden Marco LangChain: LangChain 1.0+ foundations (models, tools, retrievers, RAG chain).
DSA: U5 DSA: DP sections (2D, strings).

WEEKLY DELIVERABLE

[ ] RAG pipeline reimplemented in LangChain v1 components + RAGAS report + experiment log showing measured improvement.

SELF-CHECK: I CAN...

[ ] Explain faithfulness vs answer relevancy
[ ] Say what changed in LangChain 1.0 (create_agent, langchain-classic)
[ ] Show before/after metrics

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 19

PHASE 4 / 8 - RAG & VECTOR DATABASES

Advanced RAG: access control, routing, caching, ingestion

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Production concerns for RAG systems.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Metadata filtering, RBAC in retrieval, multi-tenant designs [Qdrant/pgvector docs on filtering | AI Engineering ch on RAG]	Implement per-user document permissions	Single Number, Number of 1 Bits, Counting Bits, Missing Number	☐ A ☐ B ☐ C
TUE	Agentic RAG and routing; self-RAG / corrective RAG concepts [LangGraph docs 'Agentic RAG' tutorial]	Router: vector vs SQL vs no-retrieval	Reverse Bits, Sum of Two Integers, Reverse Integer	☐ A ☐ B ☐ C
WED	GraphRAG and knowledge graphs overview (concept day) [Microsoft GraphRAG blog/paper | DeepLearning.AI 'Knowledge Graphs for RAG']	1-page notes: when GraphRAG is (not) worth it	Rotate Image, Spiral Matrix, Set Matrix Zeroes	☐ A ☐ B ☐ C
THU	Latency and cost: embedding cache, semantic cache, streaming, async retrieval	Add Redis semantic cache; measure latency drop	Happy Number, Plus One, Pow(x, n)	☐ A ☐ B ☐ C
FRI	Ingestion pipelines: incremental indexing, dedupe, versioning, background workers [Celery or ARQ docs]	Background ingestion worker with re-indexing on file change	Multiply Strings, Design HashMap. Redo Rotate Image	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Prepare corpus and scope for next week's capstone.			☐ R

UDEMY (optional supplement)
Learn: U11 Eden Marco LangGraph: Advanced RAG, Corrective RAG, Self-RAG, Adaptive RAG sections.
DSA: U5 DSA: DP hard problems + bit manipulation.

WEEKLY DELIVERABLE
[ ] RAG service with per-user document permissions, query router, Redis semantic cache, and an incremental ingestion worker.

SELF-CHECK: I CAN...
[ ] Explain how you enforce document-level ACLs in retrieval
[ ] Describe agentic RAG routing
[ ] Explain incremental indexing and dedupe

TRACKER
DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 20

PHASE 4 / 8 - RAG & VECTOR DATABASES

Capstone 4: Production RAG assistant

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Ship one RAG product that you would trust.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Scope + architecture: pick a real corpus (500+ docs), write the design doc and success metrics	Repo skeleton, docker compose, CI	CHECKPOINT 1: redo 5 problems you failed in W1-6 (from your mistakes list), no hints	☐ A ☐ B ☐ C
TUE	Build sprint: ingestion pipeline (parse, chunk, embed, index) with metadata	Ingest corpus; spot check 20 chunks	Redo 5 failed problems from W7-12 (trees, heaps, backtracking, graphs)	☐ A ☐ B ☐ C
WED	Build sprint: hybrid retrieval + rerank + grounded generation API with streaming	End-to-end working API + tests	Redo 5 failed problems from W13-19 (graphs, greedy, DP, bits)	☐ A ☐ B ☐ C
THU	Evals: 50+ question set, RAGAS + retrieval metrics, error analysis on 20 failures	Iterate to improve at least one metric by a measurable amount	Timed mock: 2 unseen mediums in 45 min, talk out loud. Review honestly	☐ A ☐ B ☐ C
FRI	Deploy + write-up: README with diagram + results, blog post #2	Deploy, record demo video, publish post	Timed mock: 1 medium + 1 hard in 60 min. Update pattern cheat-sheet. Count total solved (target ~150)	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** PHASE 4 RETRO: update interview-questions doc with RAG Q&As; (chunking, rerank, eval, hybrid).			☐ R

UDEMY
(optional supplement)

Learn: No Udemy course needed this week - the free resources in Block A are the best fit.
DSA: Skip new DSA videos this week: it is a checkpoint. Use U6 only to review problems you failed.

WEEKLY DELIVERABLE

[ ] PROJECT 4: Deployed RAG assistant on a real corpus (e.g. public regulations / API docs / company handbooks): hybrid + rerank, citations, ACLs, eval dashboard, README, blog post #2.

SELF-CHECK: I CAN...

[ ] Defend each design choice with a metric
[ ] Show eval before/after
[ ] Explain failure modes and mitigations

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 21

PHASE 5 / 8 - AGENTS: LANGCHAIN, LANGGRAPH, MCP

Agent fundamentals from scratch

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Understand agents without a framework: loops, tools, planning, limits.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Workflows vs agents; 5 patterns: chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer [Anthropic 'Building Effective Agents']	Notes + diagram of the 5 patterns with an example of each	Subarray Sum Equals K (prefix sum + hashmap), Continuous Subarray Sum	☐A ☐B ☐C
TUE	The agent loop with a raw API: tool calls, results, stop conditions [Anthropic 'Tool use' docs]	Write the loop in ~60 lines with 3 tools	Minimum Size Subarray Sum, Subarray Product Less Than K	☐A ☐B ☐C
WED	Tool design: clear schemas/descriptions, error messages, idempotency [Anthropic 'Writing effective tools for agents' post]	Harden tools with timeouts, retries, helpful errors	Fruit Into Baskets, Maximum Points You Can Obtain from Cards	☐A ☐B ☐C
THU	ReAct, planning, reflection, memory [ReAct paper | Lilian Weng 'LLM Powered Autonomous Agents']	Add a planning step; compare success on 10 tasks	Longest Subarray of 1's After Deleting One Element, Max Consecutive Ones III	☐A ☐B ☐C
FRI	Code execution tools + sandboxing; hard limits on steps/cost [E2B docs or Docker]	Python-exec tool inside a Docker sandbox; max-steps + cost cap	Redo Sliding Window Maximum + 2 new mediums from the 'sliding window' tag	☐A ☐B ☐C
SAT	**Optional 2-3h: review, catch-up, project polish.** Review; write 'when NOT to use an agent' note (important in interviews).			☐R

UDEMY (optional supplement)
Learn: U10 'The GIST of AI Agents' section + U12 Ed Donner Agentic Track: first agent weeks (agent loop, tools).
DSA: U6 only when stuck.

WEEKLY DELIVERABLE
[ ] 60-80 line tool-calling agent from scratch with 3 tools, a sandboxed code tool, step/cost limits.

SELF-CHECK: I CAN...
[ ] Differentiate workflow vs agent
[ ] Describe 5 workflow patterns
[ ] Explain tool design best practices

TRACKER
DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 22

PHASE 5 / 8 - AGENTS: LANGCHAIN, LANGGRAPH, MCP

LangChain v1 agents

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Use LangChain's high-level agent API and middleware properly.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	create_agent, tools, models [docs.langchain.com Python 'Agents']	Rebuild the W21 agent with create_agent	Capacity To Ship Packages Within D Days, Split Array Largest Sum - binary search on answer	☐ A ☐ B ☐ C
TUE	Middleware: summarization, human-in-the-loop, dynamic prompts, structured output [LangChain docs 'Middleware']	Add HITL approval for a risky tool + summarization	Minimum Number of Days to Make m Bouquets, Magnetic Force Between Two Balls	☐ A ☐ B ☐ C
WED	State and memory: short-term state, long-term store [LangChain docs 'Short-term memory' / 'Long-term memory']	Persistent threads with a Postgres-backed checkpointer	Next Greater Element II, Remove K Digits	☐ A ☐ B ☐ C
THU	Tracing + datasets + experiments [LangSmith docs or Langfuse docs]	Trace runs; build a 15-case dataset	Asteroid Collision, Decode String	☐ A ☐ B ☐ C
FRI	DeepLearning.AI 'Functions, Tools and Agents with LangChain' (concepts; APIs are older, follow v1 docs for code)	Re-implement its exercises in v1 style	Basic Calculator II, Valid Parenthesis String. Redo Split Array Largest Sum	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Note differences between raw-loop and framework versions: when would you pick each?			☐ R

UDEMY (optional supplement)
Learn: U10 Eden Marco LangChain: agents, tools, structured output, middleware/HITL sections.
DSA: U6 only when stuck.

WEEKLY DELIVERABLE

[ ] Your W21 agent rebuilt with create_agent: tools, human-in-the-loop middleware, summarization, structured output, traced with LangSmith or Langfuse.

SELF-CHECK: I CAN...

[ ] Explain what create_agent gives you
[ ] Explain middleware use-cases
[ ] Show a traced run

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____

_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 23

PHASE 5 / 8 - AGENTS: LANGCHAIN, LANGGRAPH, MCP

LangGraph deep dive

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Graph-based, controllable agents: state, branching, persistence, human review.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	StateGraph basics: state, nodes, edges, reducers [LangChain Academy 'Introduction to LangGraph' module 1 | LangGraph docs]	Build and run a simple multi-node graph	Binary Tree Zigzag Level Order Traversal, Path Sum II	☐ A ☐ B ☐ C
TUE	Routing: conditional edges, tool node, ReAct agent from scratch in LangGraph [Academy modules 1-2]	Re-create a tool-calling agent as a graph	Sum Root to Leaf Numbers, Flatten Binary Tree to Linked List	☐ A ☐ B ☐ C
WED	Persistence: checkpointers, threads, time travel [Academy persistence module | LangGraph docs 'Persistence']	Resume a run after process restart	Populating Next Right Pointers in Each Node, Delete Node in a BST	☐ A ☐ B ☐ C
THU	Human-in-the-loop: interrupts, approvals, editing state [Academy HITL module]	Approval flow for a destructive tool	Vertical Order Traversal of a Binary Tree, Count Complete Tree Nodes	☐ A ☐ B ☐ C
FRI	Streaming modes, subgraphs, parallelization (map-reduce) [Academy modules on parallelization/subgraphs]	Parallel research subgraphs that merge into one answer	Recover Binary Search Tree, Binary Tree Cameras (hard)	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Finish the Academy course exercises you skipped; clean repo.			☐ R

UDEMY (optional supplement)
Learn: U11 Eden Marco LangGraph: LangGraph basics, reflection and reflexion agents (verify update date first).
DSA: U6 only when stuck.

WEEKLY DELIVERABLE

[ ] LangGraph app: custom state graph with conditional routing, checkpointer, interrupts for approval, parallel branches.

SELF-CHECK: I CAN...

[ ] Explain nodes/edges/state/reducers
[ ] Explain checkpointer + thread_id
[ ] Implement an interrupt/approve flow

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 24

PHASE 5 / 8 - AGENTS: LANGCHAIN, LANGGRAPH, MCP

Multi-agent systems, memory, agent evals, reliability

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Design larger agent systems and make them measurable and reliable.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Multi-agent patterns: supervisor, handoff/swarm, hierarchical [LangGraph docs multi-agent | Anthropic 'How we built our multi-agent research system']	Supervisor + 2 worker agents	Evaluate Division, Minimum Height Trees	☐ A ☐ B ☐ C
TUE	Long-term memory: LangGraph Store, semantic memory, user profiles [LangGraph docs 'Memory' | Academy memory module]	Remember user preferences across threads	01 Matrix, Open the Lock	☐ A ☐ B ☐ C
WED	Study a real deep-research agent architecture [LangChain 'Open Deep Research' repo]	Run it; read the code; diagram the graph	Making A Large Island, Number of Closed Islands	☐ A ☐ B ☐ C
THU	Agent evals: final-answer vs trajectory vs tool-call accuracy [LangSmith evals docs | Hamel Husain on error analysis]	Eval 20 tasks; categorize failures	Critical Connections in a Network (hard, Tarjan)	☐ A ☐ B ☐ C
FRI	Reliability engineering: retries, timeouts, loop detection, cost caps, fallbacks, idempotency	Wrap the agent with reliability controls; chaos-test tool failures	Find the City With the Smallest Number of Neighbors (Floyd-Warshall). Redo Evaluate Division	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Update interview doc with agent Q&As.;			☐ R

UDEMY (optional supplement)
Learn: U11 Eden Marco LangGraph: multi-agent section. U12 Ed Donner Agentic Track: multi-agent (CrewAI / LangGraph) lessons.
DSA: U6 only when stuck.

WEEKLY DELIVERABLE

[ ] Supervisor multi-agent app + long-term memory + agent eval harness (20 tasks) + reliability wrapper.

SELF-CHECK: I CAN...

[ ] Compare supervisor vs swarm patterns
[ ] Define trajectory evals
[ ] List 5 reliability controls

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 25

PHASE 5 / 8 - AGENTS: LANGCHAIN, LANGGRAPH, MCP

MCP + other frameworks + agent security

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Build and consume MCP servers; know the framework landscape; threat-model agents.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	MCP concepts: servers, clients, tools/resources/prompts, transports [modelcontextprotocol.io intro + architecture]	Run an existing MCP server (e.g. filesystem) with a client; inspect messages	Minimum Falling Path Sum, Count Square Submatrices	☐A ☐B ☐C
TUE	Build an MCP server in Python [modelcontextprotocol.io Python SDK / FastMCP docs]	Server exposing notes and RAG-search tools	Perfect Squares, Russian Doll Envelopes	☐A ☐B ☐C
WED	Connect MCP to agents [langchain-mcp-adapters docs | an MCP-capable client]	LangGraph agent using your MCP server	Stone Game, Palindrome Partitioning II	☐A ☐B ☐C
THU	Framework tour: PydanticAI, OpenAI Agents SDK, Claude Agent SDK, CrewAI - skim each [official docs]	Re-implement one small agent in a second framework; write a comparison	Maximal Rectangle (hard), Longest String Chain	☐A ☐B ☐C
FRI	Agent security: indirect prompt injection, least privilege, secrets, audit logs [OWASP LLM Top 10 | Simon Willison on prompt injection]	1-page threat model for your agent + 2 mitigations implemented	Mixed DP: 3 unseen mediums from 'dynamic programming' tag	☐A ☐B ☐C
SAT	**Optional 2-3h: review, catch-up, project polish.** Review; polish MCP server README.			☐R

UDEMY (optional supplement)
Learn: U12 Ed Donner Agentic Track: MCP, OpenAI Agents SDK, CrewAI sections. U10: MCP section.
DSA: U6 only when stuck.

WEEKLY DELIVERABLE

[ ] Python MCP server exposing your notes/RAG as tools, connected to a LangGraph agent and an MCP client; 1-page threat model.

SELF-CHECK: I CAN...

[ ] Explain MCP tools/resources/prompts and transports
[ ] Name 3 alternative agent frameworks and trade-offs
[ ] Describe prompt injection via tool output

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 26

PHASE 5 / 8 - AGENTS: LANGCHAIN, LANGGRAPH, MCP

Capstone 5: Agent product

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Ship one agent that does real, valuable work with humans in control.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Pick the problem + success metrics; design the graph/state; threat model	Repo skeleton, tools stubbed, CI	Insert Delete GetRandom O(1), Design HashMap	☐ A ☐ B ☐ C
TUE	Build sprint: tools + graph + persistence	Happy path working end to end	LFU Cache (hard) - design-style problem	☐ A ☐ B ☐ C
WED	Build sprint: HITL, memory, error handling, cost caps	Edge cases + tests	Sliding Window Median, Smallest Range Covering Elements from K Lists	☐ A ☐ B ☐ C
THU	Evals: 20-30 tasks, trajectory + outcome metrics; fix top 3 failures	Metrics table before/after	Design Browser History, Design Parking System: practise clean class APIs	☐ A ☐ B ☐ C
FRI	Deploy + write-up: README, demo video, blog post #3	Publish and share	Design problems review + redo LRU Cache from memory in 15 min	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** PHASE 5 RETRO: refresh resume bullets with measurable results from projects 3-5.			☐ R

UDEMY (optional supplement)
Learn: No Udemy course needed this week - the free resources in Block A are the best fit.
DSA: U6 only when stuck.

WEEKLY DELIVERABLE

[ ] PROJECT 5: Deployed agent (code-review bot / data-analyst over Postgres / support agent with RAG + tools + HITL), with evals, traces, demo video, blog post #3.

SELF-CHECK: I CAN...

[ ] Show trajectory evals
[ ] Show a human approval step
[ ] Explain failure handling and costs

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____

_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 27

PHASE 6 / 8 - HUGGING FACE, FINE-TUNING & SERVING

Hugging Face ecosystem

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Work fluently with open models, datasets and the Trainer; fine-tune small models.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	HF Hub, Transformers pipelines, AutoModel/AutoTokenizer [huggingface.co/learn LLM Course ch 1-2]	Run 5 different pipelines (classify, NER, summarize, QA, generate)	AMAZON week: LeetCode company tag 'Amazon', sort by frequency (last 6 months), 2 unseen mediums	☐A ☐B ☐C
TUE	Datasets library, preprocessing, Trainer API [HF LLM Course ch 3 + 5]	Fine-tune DistilBERT on a classification dataset; push to Hub	Amazon tag: 2 more unseen mediums (45 min cap each)	☐A ☐B ☐C
WED	Generation: sampling strategies, beam search, chat templates [HF docs 'Text generation']	Sweep temperature/top-p; write findings	Amazon tag: 1 medium + 1 hard attempt	☐A ☐B ☐C
THU	Fine-tuning embedding models [sbert.net 'Training Overview']	Fine-tune on your RAG domain pairs; measure recall@k delta	Amazon tag: redo any you failed this week	☐A ☐B ☐C
FRI	Evaluating open models: benchmarks, lm-evaluation-harness overview, picking models [HF Open LLM Leaderboard docs | lm-eval-harness README]	Compare 2 open models on your own eval set	Timed: 2 Amazon-tag mediums in 50 min, then write STAR story drafts for 2 problems solved	☐A ☐B ☐C
SAT	**Optional 2-3h: review, catch-up, project polish.** Free GPU hygiene (Colab/Kaggle); review; push notebooks.			☐R

UDEMY (optional supplement)
Learn: U9 Ed Donner Core Track: Week 3 'Embracing Open-Source Models' (Hugging Face, Colab).
DSA: U6 only when stuck.

WEEKLY DELIVERABLE

[ ] DistilBERT classifier fine-tuned with Trainer + fine-tuned embedding model with measured recall@k gain on your RAG domain.

SELF-CHECK: I CAN...

[ ] Use pipeline/AutoModel/AutoTokenizer
[ ] Explain sampling params (temperature, top-p, top-k)
[ ] Show recall@k before/after embedding fine-tune

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 28

PHASE 6 / 8 - HUGGING FACE, FINE-TUNING & SERVING

Fine-tuning LLMs (SFT, LoRA/QLoRA)

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Know when to fine-tune and how to do it cheaply.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Prompt vs RAG vs fine-tune decision framework [AI Engineering ch on finetuning | HF blog posts]	Write your own decision tree (1 page)	MICROSOFT week: tag 'Microsoft', 2 unseen mediums	☐A ☐B ☐C
TUE	PEFT, LoRA, QLoRA theory [LoRA paper intro | QLoRA paper abstract | HF PEFT docs | DeepLearning.AI 'Finetuning Large Language Models']	Notes + hand-calculate parameter savings for a 7B model	Microsoft tag: 2 more unseen mediums (string/array/tree emphasis)	☐A ☐B ☐C
WED	SFT data: instruction formats, synthetic data, quality filtering	Curate 500-1000 training examples for a narrow task	Microsoft tag: 1 medium + 1 hard attempt	☐A ☐B ☐C
THU	SFT with TRL + Unsloth on Colab [TRL docs SFTTrainer | Unsloth notebooks]	Run QLoRA training on a 1-3B model	Microsoft tag: redo fails; practise explaining complexity out loud	☐A ☐B ☐C
FRI	Evaluate, merge adapters, push to Hub; DPO/preference tuning concept [TRL DPO docs]	Comparison table: base vs fine-tuned vs prompted frontier (quality, cost, latency)	Timed: 2 mediums in 50 min	☐A ☐B ☐C
SAT	**Optional 2-3h: review, catch-up, project polish.** Review what the fine-tune changed (and didn't). Update LLM interview doc.			☐R

UDEMY (optional supplement)
Learn: U9 Ed Donner Core Track: Week 6 'Fine-tuning ... with LoRA/QLoRA'.
DSA: U6 only when stuck.

WEEKLY DELIVERABLE
[ ] QLoRA fine-tune of a 1-3B open model on a narrow task + comparison table vs base and vs prompted frontier model.

SELF-CHECK: I CAN...
[ ] Explain LoRA rank/alpha and why it saves memory
[ ] State when NOT to fine-tune
[ ] Show a quality/cost comparison table

TRACKER
DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 29

PHASE 6 / 8 - HUGGING FACE, FINE-TUNING & SERVING

Inference optimisation and serving

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Understand why inference costs what it does and how to cut it.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	How inference works: prefill/decode, KV cache, batching, memory math [AI Engineering inference chapter | DeepLearning.AI 'Efficiently Serving LLMs']	Calculate memory and throughput for a 7B model on a given GPU	GOOGLE week: tag 'Google', 2 unseen mediums (graphs, strings, DP skew)	☐☐☐ A B C
TUE	Quantization: INT8/INT4, GPTQ, AWQ, GGUF [DeepLearning.AI 'Quantization Fundamentals with Hugging Face' | llama.cpp README]	Run a quantized GGUF model; compare quality/speed vs full precision	Google tag: 2 more unseen mediums; state edge cases BEFORE coding	☐☐☐ A B C
WED	vLLM: PagedAttention, continuous batching, OpenAI-compatible server [docs.vllm.ai quickstart]	Serve a small model with vLLM (Colab/cloud GPU); benchmark throughput	Google tag: 1 medium + 1 hard attempt	☐☐☐ A B C
THU	Optimisation techniques: speculative decoding, prompt caching, routing/cascades, distillation concept	Build a router: cheap model first, escalate on low confidence; evaluate	Google tag: redo fails; write alternative solutions (brute -> optimal)	☐☐☐ A B C
FRI	Cost modelling: $/1M tokens, build vs buy, latency budgets	Spreadsheet: cost of your RAG app at 10k / 1M queries per day	Timed: 1 medium + 1 hard in 60 min	☐☐☐ A B C
SAT	**Optional 2-3h: review, catch-up, project polish.** Review; write notes on 'self-host vs API' trade-offs.			☐ R

UDEMY (optional supplement)
Learn: No Udemy course needed this week - the free resources in Block A are the best fit. (vLLM and quantization are best from the docs + DeepLearning.AI short courses.)
DSA: U6 only when stuck.

WEEKLY DELIVERABLE
[ ] vLLM or Ollama-served model behind FastAPI + model router (cheap -> expensive) + cost model spreadsheet.

SELF-CHECK: I CAN...
[ ] Explain prefill vs decode and the KV cache
[ ] Explain quantization trade-offs
[ ] Estimate cost for 1M queries/day

TRACKER
DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 30

PHASE 6 / 8 - HUGGING FACE, FINE-TUNING & SERVING

Mini-project: fine-tuned and served domain model

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Close the loop: fine-tune, serve, evaluate, report.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Define the narrow task + success metric; finalize dataset	Dataset v1 + eval set (50+ items)	META week: tag 'Meta' (or Facebook), 2 unseen mediums - speed matters (about 20 min each)	☐ A ☐ B ☐ C
TUE	Train v1 with QLoRA; record training config and metrics	Training run + checkpoints on Hub	Meta tag: 2 more mediums at speed	☐ A ☐ B ☐ C
WED	Serve with vLLM/Ollama behind FastAPI; add streaming	Working endpoint + tests	Meta tag: 2 more mediums at speed; trees/graphs/arrays emphasis	☐ A ☐ B ☐ C
THU	Evaluate vs baselines; error analysis on 20 failures; iterate once	Results table + charts	Meta tag: redo fails	☐ A ☐ B ☐ C
FRI	Write-up: README + blog post #4 with honest conclusions (what worked/failed)	Publish; add to portfolio	Timed: 2 mediums in 35 min (Meta-style). Review pace	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** PHASE 6 RETRO. Skim Chip Huyen's chapters again for gaps.			☐ R

UDEMY (optional supplement)
Learn: No Udemy course needed this week - the free resources in Block A are the best fit.
DSA: U6 only when stuck.

WEEKLY DELIVERABLE
[ ] PROJECT 6: Domain model fine-tuned + served + compared on your eval set; report with quality/cost/latency; blog post #4.

SELF-CHECK: I CAN...
[ ] Defend 'fine-tune vs not' with data
[ ] Show serving benchmarks
[ ] Explain deployment trade-offs

TRACKER
DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 31

PHASE 7 / 8 - PRODUCTION, CLOUD & FLAGSHIP PROJECT

Evals in CI, observability, guardrails

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Treat LLM quality like software quality: tests, traces, metrics, safety.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Eval-driven development: offline vs online evals, regression suites, CI gates [Hamel Husain posts | LangSmith/Braintrust docs]	Run your eval set in GitHub Actions; fail on regression	INDIAN PRODUCT COS week: tags 'Flipkart', 'Swiggy', 'Uber', 'Atlassian' etc. 2 unseen mediums	☐A ☐B ☐C
TUE	Error analysis: read traces, categorize failures, prioritize fixes [Hamel Husain & Shreya Shankar on error analysis]	Review 100 traces; build a failure-mode spreadsheet; fix top 2	Same tags: 2 more mediums (expect arrays, graphs, DP, design-ish)	☐A ☐B ☐C
WED	Observability: tracing, metrics, alerts, cost dashboards [Langfuse docs | OpenTelemetry GenAI conventions]	Dashboard: latency p50/p95, cost/request, error rate	Same tags: 1 medium + 1 hard attempt	☐A ☐B ☐C
THU	Guardrails and red-teaming: moderation, PII, jailbreaks [Guardrails AI or NeMo Guardrails docs | OWASP LLM Top 10]	Red-team with 30 adversarial prompts; patch gaps	Same tags: redo fails	☐A ☐B ☐C
FRI	Feedback loops: thumbs up/down, human review queues, dataset growth	Feedback endpoint + small review UI; add reviewed cases to eval set	Timed: 2 mediums in 45 min	☐A ☐B ☐C
SAT	**Optional 2-3h: review, catch-up, project polish.** Review; update resume bullets (metrics!).			☐R

UDEMY (optional supplement)
Learn: No Udemy course needed this week - the free resources in Block A are the best fit.
DSA: U6 only when stuck.

WEEKLY DELIVERABLE

[ ] Eval suite running in CI on your RAG app + error-analysis spreadsheet + tracing dashboard + red-team report + feedback loop.

SELF-CHECK: I CAN...

[ ] Explain offline vs online evals
[ ] Describe error analysis on 100 traces
[ ] Name guardrail layers

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 32

PHASE 7 / 8 - PRODUCTION, CLOUD & FLAGSHIP PROJECT

Cloud, deployment, IaC and scaling

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Deploy like a professional: cloud basics, containers, managed services, infra as code, load testing.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Cloud basics: IAM, compute, storage, networking, billing alerts [AWS Skill Builder 'Cloud Practitioner Essentials' (free)]	YOU do this: create AWS account, MFA, budget alert, least-privilege IAM user (never put keys in code)	SPACED REVISION: redo 4 problems from your mistakes list (oldest first)	☐ A ☐ B ☐ C
TUE	Deploy a container: ECR + ECS Fargate or App Runner (or GCP Cloud Run) [AWS docs for ECS/App Runner]	Deploy your FastAPI app; health checks	Redo 4 more from the mistakes list	☐ A ☐ B ☐ C
WED	Managed services: RDS Postgres + pgvector, S3, Secrets Manager; Bedrock overview [AWS docs]	Wire app to RDS + secrets; no secrets in env files in git	Redo 4 more; add one-line insight to each in your notebook	☐ A ☐ B ☐ C
THU	Infrastructure as code: Terraform basics [HashiCorp 'Get Started - AWS' tutorials]	Codify part of the infra in Terraform; plan/apply/destroy	Re-derive from memory: LRU Cache, Dijkstra, Trie, topological sort	☐ A ☐ B ☐ C
FRI	Scaling and reliability: queues (SQS/Celery), autoscaling, load testing [Locust or k6 docs]	Load test; identify top 2 bottlenecks and note fixes	Re-derive: DP templates (knapsack, LCS, LIS), backtracking template, union-find	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Tear down costly resources! Review bill. Notes on deployment architecture.			☐ R

UDEMY (optional supplement)
Learn: U13 Maarek AWS Practitioner: cloud basics, IAM, EC2, S3, RDS, pricing sections (Mon-Wed). U4 Docker Mastery: 'Docker in Production' + 'Security'.
DSA: U6 only when stuck.

WEEKLY DELIVERABLE

[ ] Capstone-ready deployment: container on AWS (ECS Fargate or App Runner) with RDS Postgres + secrets, basic Terraform, and a load-test report.

SELF-CHECK: I CAN...

[ ] Explain IAM roles vs users
[ ] Explain container deploy flow ECR -> ECS
[ ] Show load-test results

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 33

PHASE 7 / 8 - PRODUCTION, CLOUD & FLAGSHIP PROJECT

Capstone 6a: End-to-end AI product (build)

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Your flagship project: a multi-tenant 'Docs Copilot' (RAG + agent + MCP tool + evals) deployed on cloud.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Design doc + ADRs (architecture decision records); data model; threat model [AI Engineering architecture chapter | Michael Nygard ADR post]	Repo skeleton, infra stubs, CI, ADR-001..003	HARD sampler: pick 1 hard from 'sliding window' tag; limit 40 min then study	☐ A ☐ B ☐ C
TUE	Build: ingestion + hybrid retrieval + rerank (reuse Project 4)	Multi-tenant ingestion with ACLs	1 hard from 'graph' tag	☐ A ☐ B ☐ C
WED	Build: agent layer (LangGraph) + MCP tool(s) (reuse Project 5)	Agent answers + takes actions with HITL	1 hard from 'dynamic programming' tag	☐ A ☐ B ☐ C
THU	Build: auth, API, limits, UI	End-to-end happy path in UI	1 hard from 'binary search' or 'heap' tag	☐ A ☐ B ☐ C
FRI	Evals: 80+ cases, retrieval + answer + trajectory; CI gate	Eval report v1	Redo the 2 hardest of the week from scratch	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Catch up on anything unfinished.			☐ R

UDEMY (optional supplement)
Learn: No Udemy course needed this week - the free resources in Block A are the best fit.
DSA: U6 only when stuck.

WEEKLY DELIVERABLE

[ ] Working flagship backend: ingestion, hybrid retrieval, agent layer, auth/multi-tenancy, limits, eval suite in CI.

SELF-CHECK: I CAN...

[ ] Walk through the architecture in 5 minutes
[ ] Explain trade-offs (ADRs)
[ ] Show eval metrics in CI

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 34

PHASE 7 / 8 - PRODUCTION, CLOUD & FLAGSHIP PROJECT

Capstone 6b: harden, document, portfolio

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Polish until a recruiter or interviewer can understand it in 3 minutes.

DAY	A: Learn (2h)	B: Build / practice (1h)	C: DSA (1h)	DONE
MON	Observability + cost dashboard for the flagship [Langfuse/Grafana docs]	Dashboards + alerts	Virtual contest or past LeetCode contest #1 (90 min). Review after	☐ A ☐ B ☐ C
TUE	Load test and fix top 2 bottlenecks [Locust/k6]	Before/after numbers in README	Upsolve unsolved contest problems	☐ A ☐ B ☐ C
WED	Documentation day: README, architecture diagram, ADRs, eval report	Record 3-min demo video	Virtual contest or past contest #2 (90 min)	☐ A ☐ B ☐ C
THU	Portfolio: GitHub profile README, pin top repos, blog post #5, LinkedIn update	Publish blog post #5	Upsolve; update weak-topic list	☐ A ☐ B ☐ C
FRI	Resume v1: 1 page, AI Engineer + SWE bullets with metrics (STAR) [Google 'XYZ' resume formula: accomplished X as measured by Y by doing Z]	Ask 2 people to review; fix feedback	Rest-light: re-solve 3 favourite problems; update pattern cheat-sheet. Count total (target ~280-300)	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** PHASE 7 RETRO. Next 6 weeks = interview sprint. Start applying for referrals in parallel.			☐ R

UDEMY (optional supplement)
Learn: No Udemy course needed this week - the free resources in Block A are the best fit.
DSA: U6 only when stuck.

WEEKLY DELIVERABLE

[ ] PROJECT 7 (flagship): deployed, observable, load-tested; README + architecture diagram + ADRs + eval report; demo video; blog post #5; resume v1; LinkedIn/GitHub polished.

SELF-CHECK: I CAN...

[ ] 3-minute project pitch
[ ] Defend the numbers on your resume
[ ] Explain what you'd do at 100x scale

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 35

PHASE 8 / 8 - INTERVIEW SPRINT (DSA, LLD, HLD, MOCKS)

Interview sprint 1: OOP and low-level design (LLD)

Week of: _____ / _____
Hours done: ____ / 20

GOAL: LLD is asked at Amazon, Microsoft and most Indian product companies (Zomato, Swiggy, Flipkart). Build a repeatable approach.

DAY	A: Design & CS fundamentals (1.5h)	B: Behavioral / mock / ML Qs (0.5h)	C: DSA (2h)	DONE
MON	SOLID + core design patterns (Strategy, Factory, Observer, Singleton) [Refactoring Guru 'Design Patterns' | Head First Design Patterns]	Draft 6 STAR stories from your work/projects [Amazon Leadership Principles page]	2 timed mediums (45 min each) + 30 min review: Trees (W7-8 topics)	☐ A ☐ B ☐ C
TUE	LLD: Parking Lot (requirements -> classes -> code) [Awesome Low Level Design repo (GitHub)]	ML fundamentals Q&A;: bias/variance, regularization, metrics (use your W8 cheat-sheet)	2 timed mediums + review: Graphs BFS/DFS	☐ A ☐ B ☐ C
WED	LLD: Elevator system or Splitwise	Python Q&A;: GIL, asyncio, decorators, generators, memory	2 timed mediums + review: DP	☐ A ☐ B ☐ C
THU	LLD: LRU cache / Rate limiter / BookMyShow-style seat booking	Practice explaining your Project 4 in 3 minutes	2 timed mediums + review: Heaps/Intervals	☐ A ☐ B ☐ C
FRI	LLD: Food-delivery order system (Zomato/Blinkit-style) - mock it with a timer (45 min)	Self-review recording; list 3 improvements	2 timed mediums + review: Binary search / sliding window	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Rewrite your weakest LLD from scratch; update STAR stories.			☐ R

UDEMY (optional supplement)
Learn: U15 LLD course is optional and has caveats (see index). Prefer Refactoring Guru + the LLD repo.
DSA: U6 for walkthroughs on failed problems.

WEEKLY DELIVERABLE

[ ] 5 LLD designs written in Python (classes, interfaces, patterns) + 6 STAR stories drafted.

SELF-CHECK: I CAN...

[ ] Use SOLID in a design discussion
[ ] Name 4 design patterns and a use-case each
[ ] Tell 2 STAR stories in 2 min each

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____

_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 36

PHASE 8 / 8 - INTERVIEW SPRINT (DSA, LLD, HLD, MOCKS)

Interview sprint 2: high-level system design fundamentals

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Learn the building blocks and a framework for system design interviews.

DAY	A: System design (1.5h)	B: Behavioral / mock / ML Qs (0.5h)	C: DSA (2h)	DONE
MON	Scalability basics: load balancing, caching, CDNs, DB scaling [Alex Xu 'System Design Interview' vol 1 ch 1-3 | ByteByteGo | hellointerview.com intro]	STAR stories: refine 3 using feedback from a friend	2 timed mediums + review: Strings/Hashing	☐ A ☐ B ☐ C
TUE	Databases: SQL vs NoSQL, indexing, sharding, replication, CAP/consistency [DDIA ch 5-7 skim | ByteByteGo]	Quiz: explain CAP in 90 seconds	2 timed mediums + review: Trees	☐ A ☐ B ☐ C
WED	Messaging: queues, Kafka basics, pub-sub, idempotency, retries [Alex Xu vol 1/2 relevant chapters | Kafka intro docs]	Quiz: delivery semantics	2 timed mediums + review: Graphs	☐ A ☐ B ☐ C
THU	Design: URL shortener + rate limiter (full framework, 45 min)	Peer or self-record mock; note gaps	2 timed mediums + review: DP	☐ A ☐ B ☐ C
FRI	Design: news feed / notification system / live delivery tracking (Zomato-style)	Mock with a friend (or Pramp/interviewing.io-style peer mock)	2 timed mediums + review: Backtracking	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Redo one design on a blank page without notes.			☐ R

UDEMY (optional supplement)
Learn: U14 Frank Kane 'Mastering the System Design Interview': fundamentals and first designs.
DSA: U6 for walkthroughs on failed problems.

WEEKLY DELIVERABLE
[ ] 4 designs on paper (URL shortener, rate limiter, feed/notifications, delivery tracking) using a consistent framework.

SELF-CHECK: I CAN...
[ ] Walk through requirements -> estimates -> API -> data -> HLD -> deep dive -> trade-offs
[ ] Explain sharding vs replication
[ ] Explain at-least-once and idempotency

TRACKER
DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 37

PHASE 8 / 8 - INTERVIEW SPRINT (DSA, LLD, HLD, MOCKS)

Interview sprint 3: classic HLD + AI system design

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Large-scale classics plus the AI-specific designs your target roles will ask.

DAY	A: System design (1.5h)	B: Behavioral / mock / ML Qs (0.5h)	C: DSA (2h)	DONE
MON	Design: WhatsApp-style chat (WebSockets, delivery, ordering)	Behavioral: 'conflict', 'failure', 'ownership' stories	2 timed mediums + review: Intervals/Greedy	☐A ☐B ☐C
TUE	Design: Blinkit/Uber-style dispatch + inventory	Behavioral: refine from yesterday	2 timed mediums + review: Heaps/Graphs	☐A ☐B ☐C
WED	AI design: ChatGPT-like service (LLM serving, streaming, rate limits, batching, caching) [AI Engineering inference + architecture chapters | vLLM docs]	Explain KV cache in 60 sec	2 timed mediums + review: Mixed	☐A ☐B ☐C
THU	AI design: enterprise RAG over 10M docs (ingestion, vector DB sharding, ACLs, eval, freshness)	Explain RAG trade-offs in 2 min	2 timed mediums + review: Mixed	☐A ☐B ☐C
FRI	AI design: multi-agent customer-support platform (HITL, tool safety, observability, cost control) + full mock	Self-review; list gaps	2 timed mediums + review: Mixed	☐A ☐B ☐C
SAT	**Optional 2-3h: review, catch-up, project polish.** Make a 2-page 'design cheat-sheet' (numbers, patterns, checklists).			☐R

UDEMY (optional supplement)
Learn: U14 Frank Kane: remaining classic designs (chat, feed, dispatch). AI-specific designs: use Chip Huyen + vLLM docs (no Udemy match).
DSA: U6 for walkthroughs on failed problems.

WEEKLY DELIVERABLE

[ ] 6 more designs incl. ChatGPT-like serving, enterprise RAG, and multi-agent platform.

SELF-CHECK: I CAN...

[ ] Estimate QPS/storage for a design
[ ] Design an LLM serving stack with batching/caching/streaming
[ ] Design RAG with ACLs and evals at 10M docs

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 38

PHASE 8 / 8 - INTERVIEW SPRINT (DSA, LLD, HLD, MOCKS)

Interview sprint 4: AI/ML theory + mock loop #1

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Be sharp on fundamentals and run a full simulated loop.

DAY	A: AI / ML / Python / SQL Qs (1.5h)	B: Mock interview / behavioral (0.5h)	C: DSA (2h)	DONE
MON	ML/DL theory drill: transformers, attention, tokenization, embeddings, loss, optimizers, overfitting (50 Qs) [Chip Huyen 'Introduction to Machine Learning Interviews' book (free online)]	Answer 5 Qs out loud, timed	2 timed mediums + review: Arrays/Strings	☐ A ☐ B ☐ C
TUE	LLM-specific drill: RAG trade-offs, evals, hallucination, fine-tuning, agent failure modes, cost (30 Qs)	Answer 5 Qs out loud	2 timed mediums + review: Trees	☐ A ☐ B ☐ C
WED	Python deep Qs: GIL, asyncio, memory, decorators, generators, typing, packaging	Answer 5 Qs out loud	2 timed mediums + review: Graphs	☐ A ☐ B ☐ C
THU	SQL + backend Qs: joins, indexes, transactions, caching, REST design, auth	Whiteboard 2 SQL problems	2 timed mediums + review: DP	☐ A ☐ B ☐ C
FRI	Full mock loop #1 (peer): 1 DSA (45 min) + 1 design (45 min) + behavioral (20 min). Record it	Review recording: list 5 fixes	1 hard problem (60 min cap) + study	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Fix the 5 issues from the mock; redo failed problems.			☐ R

UDEMY (optional supplement)

Learn: No Udemy course needed this week - the free resources in Block A are the best fit.
DSA: U6 for walkthroughs on failed problems.

WEEKLY DELIVERABLE

[ ] Q&A; doc of ~100 questions answered in your own words + full mock loop #1 recorded.

SELF-CHECK: I CAN...

[ ] Explain transformers/attention clearly
[ ] Compare RAG vs fine-tuning in 2 min
[ ] Pass a full mock loop with feedback

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 39

PHASE 8 / 8 - INTERVIEW SPRINT (DSA, LLD, HLD, MOCKS)

Interview sprint 5: company-specific prep + mock loop #2

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Tune for each target: Amazon (Leadership Principles + bar-raiser), Google, Microsoft, Meta, Indian product companies.

DAY	A: Company-specific prep (1.5h)	B: Mock / behavioral (0.5h)	C: DSA (2h)	DONE
MON	Amazon: Leadership Principles deep dive; map each STAR story to 2 LPs; bar-raiser style Qs [Amazon Leadership Principles page]	Say 2 stories aloud, timed (2 min)	2 timed mediums + review: Amazon-tag problems	☐ A ☐ B ☐ C
TUE	Google: communicating thought process, edge cases, complexity; brute -> optimal narration	Mock: explain one hard problem aloud	2 timed mediums + review: graphs/strings/DP	☐ A ☐ B ☐ C
WED	Microsoft and Meta: Meta speed drills (2 problems in 35 min), Microsoft collaborative style	Mock: 2 problems in 35 min	2 timed mediums + review: speed set	☐ A ☐ B ☐ C
THU	Indian product cos (Zomato, Blinkit, Swiggy, Flipkart): DSA + LLD + HLD rounds, project deep-dive; prepare 'why this company' answers	Deep-dive Q&A; on your flagship project (10 tough Qs)	2 timed mediums + review: mixed	☐ A ☐ B ☐ C
FRI	Full mock loop #2 (peer): DSA + LLD or design + behavioral	Review recording; list fixes	1 hard problem + study	☐ A ☐ B ☐ C
SAT	**Optional 2-3h: review, catch-up, project polish.** Write prep sheets for your top 5 target companies.			☐ R

UDEMY (optional supplement)
Learn: No Udemy course needed this week - the free resources in Block A are the best fit.
DSA: U6 for walkthroughs on failed problems.

WEEKLY DELIVERABLE
[ ] One-page prep sheet per target company + mock loop #2.

SELF-CHECK: I CAN...
[ ] Map your STAR stories to Amazon's 16 LPs
[ ] Know each company's loop format
[ ] Pass mock loop #2

TRACKER
DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____
_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

WEEK 40

PHASE 8 / 8 - INTERVIEW SPRINT (DSA, LLD, HLD, MOCKS)

Interview sprint 6: polish and launch applications

Week of: _____ / _____
Hours done: ____ / 20

GOAL: Convert preparation into interviews: referrals, applications, weak-spot drilling.

DAY	A: Applications & polish (1.5h)	B: Mock / behavioral (0.5h)	C: DSA (2h)	DONE
MON	Resume final, LinkedIn headline/About, GitHub pinned repos; check every number on the resume is defensible	Ask a friend to read the resume cold; fix	2 timed mediums + review: weak topic #1	☐☐☐ A B C
TUE	Target list: 30 companies; find alumni/colleagues for referrals; write outreach template [LinkedIn/alumni networks]	Send first 5 referral requests (YOU send them)	2 timed mediums + review: weak topic #2	☐☐☐ A B C
WED	Weak-area drill based on mock feedback (DSA topic, design, behavioral)	Redo worst answer from mock #1/#2	2 timed mediums + review: weak topic #3	☐☐☐ A B C
THU	Full mock loop #3: timed, 3 rounds back-to-back (DSA, design, behavioral)	Review; final fixes list	1 medium + 1 hard timed	☐☐☐ A B C
FRI	Plan next 8 weeks: apply weekly, keep DSA 1h/day, 1 new project or OSS contribution, weekly mock	Update tracker (applications, interviews, outcomes)	2 timed mediums + review: favourites	☐☐☐ A B C
SAT	**Optional 2-3h: review, catch-up, project polish.** PHASE 8 COMPLETE. Celebrate. Then keep the routine going while interviews happen.			☐ R

UDEMY
(optional supplement)

Learn: No Udemy course needed this week - the free resources in Block A are the best fit.
DSA: U6 for walkthroughs on failed problems.

WEEKLY DELIVERABLE

[ ] Final resume + LinkedIn + GitHub; list of 30 target companies; referral outreach sent; mock loop #3 passed; ongoing weekly plan.

SELF-CHECK: I CAN...

[ ] Resume reviewed by 2 people
[ ] 10+ referral requests sent
[ ] Know your next 8-week routine

TRACKER

DSA problems solved: ____
Failed (to redo): ____
Energy 1-5: ____
Notes: _____

_____

WHAT I LEARNED / BLOCKERS / QUESTIONS TO ASK

Udemy supplement index (courses verified Oct 2026)

How to use: the free/official resources in Block A stay primary. Use these Udemy courses as a SECOND explanation when a topic is not clicking, or for hands-on coding along. Do not watch a whole course end to end. Open the curriculum, use the search box for the keywords shown in each week, and watch only those lectures (1.25-1.5x). Check each course's 'Last updated' date and whether it is included in your Udemy plan: subscription plans can have a curated catalog, so a course might need separate purchase. Section names change when instructors re-record, so match by topic keywords, not section numbers.

Code	Course	Instructor	What to use it for
U1	The Complete Python Bootcamp From Zero to Hero in Python	Jose Portilla	~22 hrs. Use only as a refresher/fast-forward (1.5x). Skip absolute-beginner sections. Wks 1-3.
U2	FastAPI - The Complete Course 2026 (Beginner + Advanced)	Eric Roby, Chad Darby	Updated 11/2025. Projects: REST APIs, auth/authorization, production DBs, deployment. Wks 4-6.
U3	The Complete SQL Bootcamp	Jose Portilla	PostgreSQL + pgAdmin; SELECT/GROUP BY/JOINs/create tables/Python+Postgres. Wk 5.
U4	Docker Mastery: with Kubernetes + Swarm from a Docker Captain	Bret Fisher	~21.5 hrs. Do ONLY: containers, images, volumes, Compose, production, security, CI (GitHub Actions) lectures. Skip Swarm/Kubernetes. Wks 6, 32.
U5	Python Data Structures & Algorithms + LEETCODE Exercises	(instructor not verified)	Updated 4/2026. Animated explanations + coding exercises: linked lists, trees, heaps, graphs, sorting, recursion, DP. Use BEFORE each week's NeetCode problems.
U6	LEETCODE In Python: Algorithms Coding Interview Questions	(instructor not verified)	~29 hrs of easy-to-hard walkthroughs. Use ONLY when stuck on a problem type, after your own attempt.
U7	Machine Learning A-Z: AI, Python & R + ChatGPT Prize	Kirill Eremenko et al.	40+ hrs. Hands-on regression/classification/clustering. Skip the R parts. Wk 8.
U8	PyTorch for Deep Learning Bootcamp	Andrei Neagoie, Daniel Bourke	52 hrs, 358 lectures, rated 4.6. Use PyTorch fundamentals/workflow/classification sections. Wks 9-10.
U9	AI Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents (formerly 'LLM Engineering: Master AI, LLMs & Agents')	Ed Donner	8-week structure, refreshed Oct 2025. Wk 1 first LLM product; Wk 3 open-source models; Wk 4 evaluating models; Wk 6 LoRA/QLoRA fine-tuning; plus RAG and agents weeks. Wks 11-15, 27-28.
U10	LangChain - Agentic AI Engineering with LangChain & LangGraph	Eden Marco	19 hrs, 179 lectures, rated 4.6; re-recorded for LangChain 1.0+; covers RAG, tools, MCP. Not for beginners. Wks 16-25.
U11	LangGraph - Develop LLM powered AI agents with LangGraph	Eden Marco	7.5 hrs, 73 lectures, rated 4.5. Reflection/Reflexion agents, Corrective/Self/Adaptive RAG. CHECK its 'last updated' date: it may predate LangGraph 1.0. Wks 19, 23-24.
U12	AI Engineer Agentic Track: The Complete Agent & MCP Course	Ed Donner	OpenAI Agents SDK, CrewAI, LangGraph and MCP. Wks 21, 24-25.
U13	Ultimate AWS Certified Cloud Practitioner CLF-C02	Stephane Maarek	Use core sections only (cloud basics, IAM, EC2, S3, databases/RDS, pricing). Skip exam-only/practice-exam parts unless you want the certificate. Wk 32.
U14	Mastering the System Design Interview	Frank Kane	Instructor described as an ex-Amazon hiring manager. Wks 36-37.
U15	Low Level System Design [An interview perspective]	(instructor not verified)	OPTIONAL and use with care: C++-based, and one reviewer says SOLID/patterns are missing. Prefer Refactoring Guru + the LLD repo. Wk 35.

Resource index (everything referenced in the planner)

Python & backend
- CS50P - cs50.harvard.edu/python
- Python tutorial - docs.python.org/3/tutorial
- Fluent Python, 2nd ed. (Luciano Ramalho)
- Real Python - realpython.com
- Corey Schafer (YouTube) OOP / generators
- Exercism Python track - exercism.org
- FastAPI tutorial - fastapi.tiangolo.com/tutorial
- Pydantic - docs.pydantic.dev
- SQLAlchemy 2.0 tutorial - docs.sqlalchemy.org
- SQLBolt - sqlbolt.com; postgresqltutorial.com
- Pro Git (free) - git-scm.com/book; learngitbranching.js.org
- Docker docs 'Get started'; GitHub Actions docs

DSA
- NeetCode roadmap + NeetCode 150 - neetcode.io
- LeetCode (free problems) - leetcode.com
- Cracking the Coding Interview (reference)
- Striver A2Z sheet (optional, longer alternative)
- LeetCode SQL 50 study plan

Math, ML, DL
- 3Blue1Brown - Linear Algebra, Calculus, Neural Networks, Attention - 3blue1brown.com
- StatQuest (YouTube)
- Andrew Ng ML Specialization (Coursera)
- Kaggle Learn - kaggle.com/learn
- scikit-learn user guide
- PyTorch tutorials - pytorch.org/tutorials
- Karpathy Zero to Hero - karpathy.ai/zero-to-hero.html
- The Illustrated Transformer - jalammar.github.io

LLM engineering, RAG, agents
- AI Engineering (Chip Huyen, O'Reilly) - main reference book
- Anthropic docs + Building Effective Agents (anthropic.com/engineering)
- OpenAI Cookbook - cookbook.openai.com
- DeepLearning.AI short courses - deeplearning.ai/short-courses
- LangChain RAG From Scratch (YouTube)
- LangChain Academy (LangGraph) - academy.langchain.com
- LangChain/LangGraph docs - docs.langchain.com
- Hugging Face LLM + Agents courses - huggingface.co/learn
- Model Context Protocol - modelcontextprotocol.io
- Eugene Yan and Hamel Husain blogs (evals, patterns)
- OWASP Top 10 for LLM Applications

Vector DBs, eval, serving
- pgvector README - github.com/pgvector/pgvector
- Qdrant docs - qdrant.tech/documentation
- Chroma docs; FAISS wiki; Pinecone Learning Center
- RAGAS - docs.ragas.io; Langfuse - langfuse.com/docs; LangSmith docs
- vLLM - docs.vllm.ai; Ollama; llama.cpp
- TRL, PEFT, Unsloth docs; LoRA and QLoRA papers

Interview prep
- System Design Interview vols 1-2 (Alex Xu); ByteByteGo
- Designing Data-Intensive Applications (Kleppmann)
- hellointerview.com; Gaurav Sen (YouTube)
- Refactoring Guru design patterns; Awesome Low Level Design (GitHub)
- Amazon Leadership Principles (amazon.jobs)
- Chip Huyen - Introduction to Machine Learning Interviews (free online)